

Payment Frontend — Implementation Plan

api/payments.js → Payments pages → <PaymentPanel>

Service: Payment Wallet (:8083 · /payments) **Date:** 2026-06-13 **Pairs with:** Anirban (Product-Order FE)

1

api/payments.js

One module, five calls. The data layer every page and the panel use.

2

Payments pages

CustomerPayments & AdminPayments wired to history/lookup.

3

<PaymentPanel>

The shared pay step: wallet first, then Razorpay.

✓ Prerequisites already done (the infra layer)

✓ Vite proxy → :8083

✓ Relative base URL

✓ unwrap() helper

✓ getUserId() stub

How to read this document

Each phase is a self-contained chunk you can ship on its own. Green = ready to build now. Amber = needs a teammate answer first. Red = blocked. The two payment **flows** (wallet & Razorpay) and the **seam** with Anirban's checkout are drawn out on pages 4–5.



Where this fits

THE BIG PICTURE BEFORE THE CODE



React (browser)

Your pages + `<PaymentPanel>`
call relative paths



Vite proxy

`/payments` → `:8083`
(done ✓)



Payment Wallet

Spring Boot `:8083`
`/payments/*`

You're starting from a clean base

The infra phase already shipped: components call `/payments/...` (no ports), the proxy routes them to `:8083`, `unwrap()` normalizes envelopes, and `getUserId()` returns the customer's **USER-1001** id. Everything below builds on that.

The three things you'll build, and how they connect

The checkout seam — your boundary with Anirban

Anirban: `createOrder()` → `orderId`



You: `<PaymentPanel>` pays



`onSuccess` → refresh history

He never calls `/payments/*`; you never call `/orders/*`. The only thing crossing the line is the four props (next pages) and two callbacks.

⚠ One question shapes the whole panel — R4

If `POST /orders/create` with **WALLET** already charges the wallet server-side, then `<PaymentPanel>` becomes **Razorpay-only** and wallet checkout just confirms. Confirm with arka before wiring the wallet path.

1

Phase 1 — api/payments.js

FIVE FUNCTIONS · THE DATA LAYER

POST /payments/process

processWalletPayment()

REQUEST

```
{ userId, orderId, amount, idempotencyKey }
```

RESPONSE (RAW)

```
PaymentResponse { paymentId, status, message }
```

Generate **idempotencyKey** per attempt with

```
crypto.randomUUID()
```

so retries can't double-charge.

GET /payments/{paymentId}

getPayment()

REQUEST

```
- (path param)
```

RESPONSE (RAW)

```
PaymentDetailsResponse { paymentId, orderId, userId, amount, status }
```

Powers **AdminPayments** "search by id" and the order-detail view.**GET** /payments/user/{userId}

getUserPayments()

REQUEST

```
- (path param = USER-1001)
```

RESPONSE (RAW)

```
List<PaymentDetailsResponse>
```

Powers **CustomerPayments** history (and Admin's per-user list).**POST** /payments/razorpay/create-order

createRazorpayOrder()

REQUEST

```
{ userId, orderId, amount }
```

RESPONSE

```
RazorpayOrderResponse { razorpayOrderId, key, amount, currency }
```

The **key** for the checkout modal comes back here — no separate config call (confirm via R6).**POST** /payments/razorpay/verify

verifyRazorpayPayment()

REQUEST

```
{ razorpayOrderId, razorpayPaymentId, razorpaySignature, userId, orderId }
```

RESPONSE

```
success / fail
```

Called from the Razorpay handler. Verifies the signature server-side and marks the payment successful.

Envelope reminder

Payment endpoints return **raw** bodies (no {success,message,data}). `unwrap()` passes them through untouched — so call it anyway and your code stays identical to Anirban's.

2

Phase 2 — Payments pages

WIRE THE THREE SCREENS

**CustomerPayments** `pages/customer/CustomerPayments.jsx`

READY

Auto-load `getUserPayments(getUserId())` into a table on mount. Drop the manual USER-1001 input — read it from the session.

**AdminPayments** `pages/admin/AdminPayments.jsx`

READY

Wire the search box → `getPayment(paymentId)` ; show the result card. Optional: per-user list via `getUserPayments` .

**CustomerWallet** `pages/customer/CustomerWallet.jsx`

BLOCKED • R5

Balance display needs a wallet-balance endpoint that **doesn't exist in the spec**. Keep the stub until R5 is answered.

3

Phase 3 — <PaymentPanel> • flow A

THE WALLET PATH

● **Wallet payment — inside <PaymentPanel> when paymentMethod = "WALLET"**

- 1 Generate `idempotencyKey = crypto.randomUUID()` for this attempt.
- 2 Call `processWalletPayment({ userId, orderId, amount, idempotencyKey })` .
- ✓ **Success** → `onSuccess(result)` . **Failure** → `onFailure(error)` .

⚠ **R4 gate**

If order-create already pays the wallet server-side, skip steps 1–2 — the panel only confirms for wallet and the full flow above applies to Razorpay only.

Razorpay payment — when paymentMethod = "RAZORPAY"

- 1 createRazorpayOrder({ userId, orderId, amount }) → { razorpayOrderId, key, amount, currency } .
- 2 Inject checkout.razorpay.com/v1/checkout.js once (resolve a promise on load).
- 3 Open modal: new Razorpay({ key, order_id: razorpayOrderId, amount, currency, handler, modal.ondismiss }) .
- 4 User pays → handler receives razorpay_order_id / razorpay_payment_id / razorpay_signature .
- 5 Map snake_case → camelCase, then verifyRazorpayPayment({ razorpayOrderId, razorpayPaymentId, razorpaySignature, userId, orderId }) .
- ✓ Verified → onSuccess . Dismissed/cancelled → onFailure , order stays PENDING.

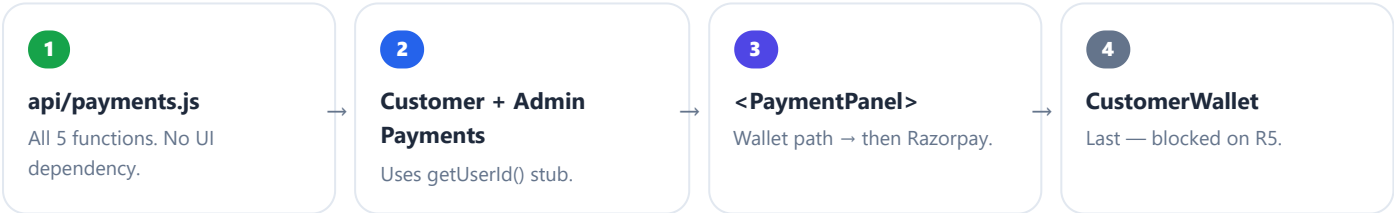
The seam contract — <PaymentPanel> props (don't change unilaterally)

PROPS IN (FROM ANIRBAN)	CALLBACKS OUT
userId // USER-1001	onSuccess(result) // confirmed / verified
orderId // from createOrder	onFailure(error) // failed / cancelled
amount // number	
paymentMethod // "WALLET" "RAZORPAY"	



Build order & blockers

SHIP IN THIS SEQUENCE



Blocker	What it blocks / what to ask arka	Workaround
R3	The USER-1001 mapping endpoint. Until it exists, getUserId() returns null.	Manual input → setUserId()
R5	No wallet-balance endpoint in the spec → CustomerWallet can't show a balance.	Keep stub
R6	Razorpay: is amount in paise or rupees? Is key always returned by create-order?	Build wallet path first
R4	Does order-create with WALLET pay server-side? Decides if the panel's wallet path runs at all.	Confirm before wiring